

Profiling from Zero

A Complete Hands-On Guide to perf, Nsight, gprof,
Valgrind, VTune, and beyond

Prepared for CK | IIT Delhi | Nanopore Project
Prof. Kolin Paul | Summer 2025

Batch 2 · The Valgrind Suite and gprof · Chapters 6 to 10

Contents

Chapter 6 Valgrind Architecture: What It Actually Does

Chapter 7 Cachegrind: Exact Cache Simulation

Chapter 8 Callgrind: Call Graph Profiling and Instruction Counting

Chapter 9 Memcheck, Massif, and Helgrind: Correctness and Memory

Chapter 10 gprof: Instrumentation-Based CPU Profiling

This batch assumes Chapters 1 to 5: hardware counters, perf stat and record, flame graphs, the roofline model, and the five benchmark programs.

CHAPTER 6

Valgrind Architecture

We are switching tools and, more importantly, switching philosophies. Everything in Batch 1 was built on one idea: the hardware counts events for free, and the profiler just reads the meter. `perf` has essentially zero overhead because the silicon was always counting. Valgrind is the opposite of this in every respect. It does not read hardware counters at all. It does not trust the hardware. It builds its own software model of a CPU and runs your program inside that model, instruction by instruction, watching everything. The overhead is enormous. The payoff is that Valgrind sees things no hardware counter can ever see, and it sees them *exactly*, with no sampling, no statistics, no skid.

I want to motivate why you would ever accept a fifty-times slowdown before I explain the machinery. Imagine you are debugging Kraken-2's cache behavior on the lab WSL2 machine. You run `perf stat` and ask for LLC miss numbers, and `perf` says *<not supported>* because Hyper-V hides the LLC counters. The hardware truth is locked away from you. What do you do? You stop asking the hardware and you ask a simulator instead. Valgrind's Cachegrind tool simulates a cache, feeds every one of your program's memory accesses through it, and tells you the exact hit and miss count at every cache level. It does not need the hardware counters because it computes the answer itself. That single capability is why Valgrind earns its place in your toolkit even though it is slow.

The Core Distinction

`perf` observes real hardware doing real work, fast and statistically. Valgrind simulates a model of hardware, slow and exactly. You use `perf` to find *where* to look. You use Valgrind to understand *exactly* what is happening at the spot `perf` pointed you to, or when the hardware refuses to tell you.

6.1 What Valgrind Actually Is

Valgrind is a **dynamic binary instrumentation framework**. Let me unpack all three words, because each one carries weight.

Dynamic means it works at run time, on the already-compiled binary, not at compile time. You do not recompile your program for Valgrind the way you do for `gprof`. You take your existing binary, exactly as it would run in production, and you launch it under Valgrind. This is a real advantage: you profile the same machine code that actually runs, including all the compiler optimizations.

Binary means it operates on machine code, not source code. Valgrind reads the actual x86 (or ARM) instructions of your program. It does not need your source, though it uses debug info (the `-g` flag) to map results back to source lines for you.

Instrumentation means it inserts extra code. As your program runs, Valgrind intercepts each block of machine instructions, adds its own bookkeeping instructions in between yours, and runs the combined result. The bookkeeping is what does the measuring: checking whether memory is valid, counting cache accesses, recording call edges, whatever the active tool needs.

The framework itself is just the engine that does this interception and instrumentation. The actual measuring is done by a *tool* that plugs into the framework. Memcheck is a tool. Cachegrind is a tool. The framework provides the universal service of grabbing every instruction and handing it to the tool; the tool decides what to do with it.

6.2 How the Instrumentation Engine Works

Here is the mechanism, because understanding it explains both the power and the slowness. When you run a program under Valgrind, your program does not execute directly on the CPU. Not a single one of your original instructions runs natively. Instead, Valgrind reads your machine code in small chunks called basic blocks, translates each block into an intermediate representation called VEX, lets the active tool add instrumentation to the VEX, translates the instrumented VEX back into real machine code, and then runs *that* on the actual CPU. This translated, instrumented code is cached so that a block executed a million times is only translated once.

Think of it like a simultaneous interpreter at the United Nations. The speaker (your program) does not address the audience directly. Every sentence goes through the interpreter (Valgrind), who hears it, possibly annotates it, and re-speaks it. The audience (the CPU) only ever hears the interpreter's version. Nothing reaches the CPU without passing through the interpreter first. That is why Valgrind sees absolutely everything, and also why everything is slow: every sentence is spoken twice and processed in between.

This disassemble-instrument-resynthesize cycle is the deepest reason Valgrind can do what perf cannot. Because every memory access passes through Valgrind's hands before reaching the CPU, Valgrind can intercept it, run it through a simulated cache, record the result, and only then let it proceed. perf can never do this; perf only sees aggregate counter totals after the fact. Valgrind sees each access individually as it happens.

Key Insight

Valgrind's slowness and Valgrind's power are the same fact viewed from two sides. It intercepts every single instruction. That interception is why it can simulate a cache exactly or catch a use-after-free precisely. It is also why your program runs ten to fifty times slower. You are paying for total visibility with total overhead. There is no free lunch; this is just the opposite trade from perf.

6.3 The Overhead Reality

Let me give you concrete numbers so you set your expectations correctly and do not panic the first time you run it.

- **Memcheck** (memory error detection), the default tool, slows programs by roughly 10x to 30x. It checks the validity and addressability of every memory access, which is a lot of bookkeeping per access.
- **Cachegrind** (cache simulation) slows programs by roughly 20x to 100x. On top of running the instrumented code, it feeds every memory access through a software cache model with full hit/miss logic.
- **Callgrind** (call graph plus cache) is similar to or worse than Cachegrind, since it does everything Cachegrind does plus tracks the entire call graph.
- **Massif** (heap profiling) is lighter, often 2x to 10x, because it only intercepts allocation calls, not every instruction's memory access.
- **Helgrind** (race detection) can be 20x to 100x because it tracks every memory access and every lock operation across all threads.

Translate this to our benchmarks. Our naive matmul takes about 8 seconds under perf. Under Cachegrind, expect 8 seconds times 50, which is roughly 7 minutes. That is fine for a one-time forensic analysis. It is completely unacceptable for repeated runs or for production. This single fact dictates how you use Valgrind: you reach for it deliberately, for a specific deep question, run it once, capture the exact answer, and put the tool away.

How This Connects to Your Project

Kraken-2 classifying a real dataset can run for many minutes natively. Under Cachegrind that becomes hours. You do not run Cachegrind on a full Kraken-2 dataset. You run it on a tiny input, a few thousand reads, just enough to exercise the hash-lookup hot loop, capture the exact LLC miss rate per function, and stop. The simulated miss rate on a small input is representative of the large input because the access pattern (random probing of the hash table) is the same regardless of how many reads you push through.

Dorado is even worse for Valgrind, because Valgrind cannot follow code onto the GPU at all. It instruments CPU instructions only. Running Dorado under Valgrind profiles the host-side orchestration and tells you nothing about the kernels. For Dorado you stay with Nsight. Valgrind is a CPU-side tool, full stop, and on this project that means it is a Kraken-2 tool.

6.4 The Tool Suite

Valgrind ships as a framework with a family of tools. You select one with `--tool=`. Here is the whole family and what each one is for, so you know which chapter is coming and why.

Memcheck: the memory error detector

The default and most famous tool. Memcheck tracks, for every byte of memory, whether it is addressable (are you allowed to touch it) and whether it is defined (does it hold a value you actually wrote). With that shadow state it catches use-after-free, buffer overflows, reads of uninitialized memory, invalid frees, and memory leaks. It is the reason most people first install Valgrind. Chapter 9 covers it.

Cachegrind: the cache simulator

Simulates a two-level cache hierarchy (first-level instruction and data caches, plus a last-level unified cache) and reports exact hit and miss counts at each level, attributable down to the source line. This is your WSL2 lifeline for the LLC numbers perf cannot read. Chapter 7, next.

Callgrind: the call-graph profiler

An extension of Cachegrind that additionally records the full call graph: which function called which, how many times, and how many instructions and cache events occurred along each call edge. Paired with the KCachegrind GUI it gives you the most detailed call-level picture available on the CPU. Chapter 8.

Massif: the heap profiler

Tracks heap allocation over time. It answers the question "how did my memory usage grow, and which call sites allocated the most?" It produces a graph of heap size against time and a breakdown of allocations by call stack. Essential for understanding Kraken-2's 180 GB database allocation. Chapter 9.

Helgrind and DRD: the thread error detectors

Both detect threading bugs: data races (two threads touching the same memory without synchronization), lock-ordering problems that can deadlock, and misuse of the POSIX threads API. Helgrind and DRD use different internal algorithms and sometimes catch different bugs; they are complementary. Relevant for the OpenMP matmul. Covered briefly in Chapter 9.

There are a couple of lesser tools (the heap profiler DHAT, the branch-prediction simulator built into Cachegrind), but the six above are the ones that matter. The pattern to internalize: *one framework, many tools, you pick the tool for the question*.

6.5 How Valgrind Differs From perf, Precisely

Let me make the perf-versus-Valgrind distinction sharp, because choosing the wrong one wastes hours, and beginners reach for the wrong one constantly.

- **Measurement method.** perf reads real hardware performance counters. Valgrind simulates execution in software and computes the answer itself. perf measures your actual CPU; Valgrind measures an idealized model CPU.
- **Accuracy.** perf is statistical (sampling) or aggregate (counters), with skid and rounding. Valgrind is exact: it counts every event with no sampling and no approximation. If Cachegrind says 687,432,109 cache misses, it means exactly that number occurred in its model.
- **Overhead.** perf is near zero. Valgrind is 10x to 100x. This alone often decides the choice.
- **Reality.** perf measures the real cache, the real branch predictor, the real prefetcher, all of which are dizzyingly complex. Valgrind's cache model is a simplified textbook cache with no prefetcher and a fixed replacement policy. So Valgrind's miss counts are exact *for its model*, which is close to but not identical to your real hardware. perf is the ground truth of your actual chip.

- **Reproducibility.** perf results vary run to run because the real machine has noise, other processes, thermal throttling. Valgrind results are deterministic: the same input gives the same counts every time, because the simulation is deterministic.

So here is the rule, and it is worth memorizing. Use **perf** to find where to look and to get real-hardware truth, fast. Use **Valgrind** when you need exact, reproducible, line-attributed counts, or when you need to see something the hardware will not show you (memory bugs, or LLC counters on WSL2). The two are complementary, not competing. A typical workflow uses perf first to localize, then Cachegrind on the localized hot region for the exact cache breakdown.

Common Pitfall

Treating Valgrind's cache miss numbers as the literal truth of your hardware. They are the truth of Valgrind's *model* of a cache, which assumes a specific size, associativity, and line length, and which has no hardware prefetcher. The prefetcher absence matters: a stride-1 access pattern that your real CPU handles beautifully via prefetching will still show some misses in Cachegrind, because the model has no prefetcher to hide them. The *relative* comparison (naive versus ikj) is rock solid. The *absolute* miss rate is model-specific. Always report Cachegrind numbers as simulated, and prefer perf's numbers when the hardware can give them.

6.6 Installation

Valgrind is in every distribution's package manager. The GUI viewer, KCachegrind, is a separate package and worth installing now because we will use it in Chapters 7 and 8.

```
# Ubuntu / Debian (including WSL2 Ubuntu)
sudo apt update
sudo apt install valgrind kcachegrind

# Fedora / RHEL
sudo dnf install valgrind kcachegrind

# Arch
sudo pacman -S valgrind kcachegrind

# verify
valgrind --version           # expect valgrind-3.xx.x
```

bash

A note on WSL2 specifically, since that is the lab machine. Valgrind works perfectly under WSL2. This is precisely why it is the workaround for the missing LLC counters. The Hyper-V virtualization that hides hardware counters from perf does not affect Valgrind at all, because Valgrind does not use hardware counters. It simulates everything in software, and software simulation does not care what the hypervisor exposes. So on WSL2 you have a slightly awkward but complete picture: perf for IPC, branches, L1, page faults; Cachegrind for the LLC numbers perf cannot reach. Between the two you get everything.

KCachegrind needs an X server to display its GUI. On WSL2 with a recent Windows build, WSLg provides this automatically and KCachegrind just opens. On older setups or over SSH you need an X server (VcXsrv on Windows, or X forwarding with `ssh -X`). If the GUI is troublesome, every KCachegrind insight is also available from the command-line annotate tools, which we will lean on heavily so the analysis never depends on a working display.

6.7 The Output File Convention

One practical detail that trips up everyone exactly once, so let me front-load it. The Valgrind tools that produce data files (Cachegrind, Callgrind, Massif) write a file named with the process ID appended, like `cachegrind.out.12345` or `massif.out.12345`. The number is the PID of the run, and it is different every time. The annotate tools (`cg_annotate`, `callgrind_annotate`, `ms_print`) take that exact filename as an argument.

```
# run produces cachegrind.out.<PID>
valgrind --tool=cachegrind ./matmul_ikj

# you must pass the SAME pid-suffixed filename to the annotator
cg_annotate cachegrind.out.12345

# a robust idiom: pin the output name so you do not have to find the PID
valgrind --tool=cachegrind --cachegrind-out-file=cg.out ./matmul_ikj
cg_annotate cg.out
```

bash

I always use the explicit `--cachegrind-out-file` (and the equivalent for the other tools) so I never have to hunt for a PID. It also makes scripts reproducible and makes before/after comparison trivial, because I can name the files *before.out* and *after.out* deliberately. Get in this habit now and you will never lose a Valgrind output file again.

What We Just Learned

Valgrind is a dynamic binary instrumentation framework. It intercepts every machine instruction at run time, translates it through an intermediate representation, lets a tool add bookkeeping, and runs the result. Nothing reaches the CPU without passing through Valgrind first.

That total interception is why Valgrind sees everything exactly and why it is 10x to 100x slower than native. Slowness and power are the same fact.

The framework hosts a family of tools: Memcheck for memory errors, Cachegrind for cache simulation, Callgrind for call graphs, Massif for heap growth, Helgrind and DRD for thread races. You pick the tool for the question.

perf reads real hardware fast and statistically. Valgrind simulates an idealized CPU slowly and exactly. Use perf to find where; use Valgrind for exact line-attributed counts or for things the hardware hides.

On WSL2, Valgrind is unaffected by the Hyper-V counter restrictions, which makes Cachegrind the workaround for the LLC numbers perf reports as not supported.

Cachegrind's model has no hardware prefetcher and a fixed cache geometry, so its absolute miss rates are model-specific. Relative comparisons are reliable; report absolute numbers as simulated.

Before You Move On

Make sure `valgrind --version` runs and that you have `kcachegrind` installed. We use the GUI in the next two chapters, though every insight has a command-line equivalent.

Internalize the output-file convention. Use `--cachegrind-out-file` (and its siblings) to name your output explicitly. You will thank yourself when you are comparing a before-cache and after-cache run of Kraken-2 and need the two files side by side.

Next chapter is `Cachegrind`, the tool that turns the abstract cache hierarchy of Chapter 2 into exact, per-line miss counts on our benchmark suite, and gives you the WSL2 LLC numbers for the Kraken-2 report page.

CHAPTER 7

Cachegrind: Exact Cache Simulation

In Chapter 2 we reasoned about caches abstractly. We argued that naive matmul thrashes the cache because of stride- n access to B , and we predicted a high miss rate. In Chapter 3 `perf` confirmed it with a real-hardware LLC miss rate of 46 percent. Now Cachegrind lets us do something neither of those could: see the exact miss count, at every cache level, attributed to the precise line of source code that caused it, deterministically, the same every run. This is the microscope. `perf` was the survey.

And there is the practical reason we need it at all. On the lab WSL2 machine, `perf` reports the LLC counters as *<not supported>*. The hardware truth is locked behind Hyper-V. Cachegrind does not ask the hardware. It simulates the cache in software and computes the miss counts itself. So when `perf` cannot give you the Kraken-2 LLC miss rate, Cachegrind can. This chapter is both a microscope for understanding and a workaround for a real limitation you will hit on the actual hardware.

7.1 What Cachegrind Simulates

Cachegrind models a machine with two levels of cache, which it calls **I1**, **D1**, and **LL**. The names matter, so let me define each precisely.

- **I1** is the first-level instruction cache. It holds the machine instructions being executed. Every instruction fetch is a lookup here.
- **D1** is the first-level data cache. It holds the data your instructions read and write. Every memory load and store is a lookup here.
- **LL** is the last-level cache, unified, meaning it holds both instructions and data. It is the backstop before main memory. A miss in **LL** is a trip to DRAM, the expensive 200-cycle event from Chapter 2.

Real modern chips have three levels (**L1**, **L2**, **L3**), but Cachegrind collapses this to two (a first level and a last level) because that two-level model captures the behavior that matters for almost all analysis: did the access hit close (**D1**), or did it fall through to the slow backstop (**LL**), or did it miss even that and go to DRAM? The middle **L2** level rarely changes the qualitative story, so the model omits it for simplicity. This is a deliberate, reasonable approximation, and it is exactly the kind of model-versus-reality gap I warned you about in Chapter 6.

By default Cachegrind reads your real CPU's cache geometry from the hardware and configures its model to match. So out of the box it simulates caches the same size and associativity as your actual machine. You can override this to model any cache you like, which is occasionally useful for answering "what if my **L1** were twice as big?"

7.2 Running Cachegrind and Configuring the Cache

The simplest invocation uses your machine's detected cache geometry:

```
gcc -O2 -g -o matmul_naive matmul_naive.c # -g for source mapping
valgrind --tool=cachegrind \
  --cachegrind-out-file=naive.cg \
  ./matmul_naive
```

bash

To pin the cache geometry explicitly, which makes results reproducible across different machines (important when you run on sauron, on LUNA, and on your laptop and want comparable numbers), specify each level as size, associativity, line-size:

```
valgrind --tool=cachegrind \
  --I1=32768,8,64 \
  --D1=32768,8,64 \
  --LL=8388608,16,64 \
  --cachegrind-out-file=naive.cg \
  ./matmul_naive
```

bash

Read those numbers. `--D1=32768,8,64` means a 32 KB first-level data cache, 8-way set associative, with 64-byte lines. `--LL=8388608,16,64` means an 8 MB last-level cache, 16-way associative, 64-byte lines. These are typical values for a modern desktop core. The 64-byte line is the cache-line size from Chapter 2, the atom of memory traffic, now appearing as an explicit simulation parameter.

7.3 Reading the Summary Output

When Cachegrind finishes, it prints a summary to stderr. Here is what the naive matmul produces, with realistic numbers:

```
==12345== Cachegrind, a cache and branch-prediction profiler
==12345== Command: ./matmul_naive
==12345==
==12345== I   refs:      9,512,008,114
==12345== I1  misses:      1,204
==12345== LLi misses:      1,142
==12345== I1  miss rate:      0.00%
==12345== LLi miss rate:      0.00%
==12345==
==12345== D   refs:      4,303,917,442 (3,224,118,003 rd + 1,079,799,439 wr)
==12345== D1  misses:      1,082,114,909 (1,081,002,433 rd + 1,112,476 wr)
==12345== LLd misses:      691,204,338 ( 690,447,219 rd + 757,119 wr)
==12345== D1  miss rate:      25.1% ( 33.5% + 0.1% )
==12345== LLd miss rate:      16.1% ( 21.4% + 0.1% )
==12345==
==12345== LL refs:      1,082,116,113 (1,081,003,637 rd + 1,112,476 wr)
==12345== LL misses:      691,205,480 ( 690,448,361 rd + 757,119 wr)
==12345== LL miss rate:      5.0% ( 5.3% + 0.1% )
```

output

This wall of numbers is intimidating the first time. Let me show you how to read it in thirty seconds. There are three blocks: instructions (I), data (D), and last-level (LL). You almost always care about the data block and the LL block; the instruction block is near-perfect for any sane program because code has excellent locality.

The instruction block. I refs is 9.5 billion instruction fetches. I1 misses is 1,204. The instruction cache miss rate is 0.00 percent. This is normal and boring. Your hot loop is a handful of instructions that fit trivially in I1 and execute billions of times. Instruction caching is essentially never your problem in numerical code. Glance at it, confirm it is near zero, move on.

The data block. This is the story. D refs is 4.3 billion data accesses, split into 3.2 billion reads and 1.1 billion writes. D1 misses is 1.08 billion, almost all of them reads. The D1 read miss rate is 33.5 percent. One in three first-level data reads misses. That is catastrophic, and it is the B-column access we have been chasing since Chapter 2, now quantified exactly: a third of all reads miss the first-level cache.

The LL block. Of the accesses that missed D1 and fell through to the last level, how many also missed LL and went to DRAM? LL misses is 691 million, the LL miss rate is 5.0 percent overall. So the picture is: about a third of reads miss L1, but most of those are caught by the 8 MB last-level cache (because B, at 8 MB, mostly fits there after the first pass), and only 5 percent of all accesses fall all the way through to DRAM. That 5 percent of 4.3 billion accesses is still 691 million DRAM round-trips, each costing 200 cycles, which is where the 8 seconds of runtime goes.

Reading the Output in One Breath

Three blocks: instructions, data, last-level. Instructions are always near-perfect, glance and move on. Data D1 miss rate tells you first-level locality (naive: 33.5 percent reads, terrible). LL miss rate tells you how often you hit DRAM (naive: 5 percent of all accesses, which is 691 million trips). The read column is almost always the one that matters for numerical code, because writes are buffered and forgiving while reads stall the pipeline.

7.4 cg_annotate: Down to the Source Line

The summary tells you the program's totals. The real power of Cachegrind is attributing those misses to exact source lines, which is what `cg_annotate` does. It reads the output file and prints your source with per-line counts of every event Cachegrind tracked.

```
cg_annotate naive.cg
```

bash

```
# or annotate a specific source file with full line-by-line detail:
cg_annotate --auto=yes naive.cg matmul_naive.c
```

The annotated inner loop of naive `matmul` looks like this. The columns are, in order: instruction reads (Ir), data reads (Dr), data writes (Dw), D1 read misses (D1mr), D1 write misses (D1mw), LL

read misses (DLmr), LL write misses (DLmw).

```

      Ir      Dr      Dw      D1mr      D1mw      DLmr      DLmw      output
      .      .      .      .      .      .      .      .
2,148,532,224 1,074,266,112 0      1,073      0      1,032      0      for (int k = 0; k < n; k++)
      \      \      \      \      \      \      \      C[i*n+j] += A[i*n+k] * B[k*n+j];
      \      \      \      \      \      \      \      \__ A access: stride-1, almost all hits
      \      \      \      \      \      \      \      \__ B access: stride-n, misses on nearly every iteration

```

Let me read this for you, because the layout is dense. The inner-loop line shows 1.07 billion data reads (Dr). Those reads are A and B. Now look at the D1mr column, the first-level data read misses. The number is enormous, on the order of the B-access count, because every stride-n access to B misses D1. The A access, being stride-1, contributes almost nothing to the miss column. `cg_annotate` has pinned the entire performance problem to a single source line and, with the column breakdown, told you it is the read misses that hurt, exactly as Chapter 2 reasoned.

Now run the same annotation on the `ikj` version:

```

gcc -O2 -g -o matmul_ikj matmul_ikj.c
valgrind --tool=cachegrind --cachegrind-out-file=ikj.cg ./matmul_ikj
cg_annotate ikj.cg

```

The same inner-loop line now shows a D1 read miss count smaller by a factor of roughly eight. Both A and B are stride-1, so each cache line is reused for eight consecutive doubles before a new line is needed. The miss count drops from one-per-iteration to one-per-eight-iterations. That factor of eight in the miss column is the entire optimization, visible as a single number, attributed to a single line, deterministically. This is the clarity Cachegrind buys you that no sampling profiler can match.

And Here Is Where It Gets Beautiful

`perf` told you the program had a high miss rate. `perf annotate` told you which instruction stalled. But Cachegrind's `cg_annotate` tells you the *exact integer count* of misses on that line, and lets you watch that integer drop by exactly 8x when you fix the loop order, every single run, with no statistical noise. When you need to prove an optimization worked, not argue it probably did, this exact reproducible integer is the strongest evidence there is.

7.5 The N-cubed Versus N-squared Story

Run all four CPU benchmarks through Cachegrind and a deeper pattern emerges that is worth seeing explicitly, because it teaches you to predict cache behavior from algorithmic structure.

```
for v in naive ikj tiled; do
  gcc -O2 -g -o matmul_$v matmul_$v.c
  valgrind --tool=cachegrind --cachegrind-out-file=$v.cg ./matmul_$v 2>&1 \
    | grep -E "D1 miss rate|LL miss rate"
done
```

bash

The naive version's LL misses scale with N-cubed, because B is effectively re-read from memory on every one of the N-squared output elements times N inner iterations, with no reuse. The ikj and tiled versions reduce LL misses toward N-squared, the theoretical minimum where each matrix element is brought from DRAM essentially once. The tiled version gets closest to N-squared because blocking maximizes reuse of each loaded tile while it sits in cache. Watching the LL miss count fall from N-cubed-ish toward N-squared-ish across the three versions is watching the arithmetic-intensity argument from Chapter 5 play out in concrete integers. The algorithm's true arithmetic intensity is being recovered by reducing redundant DRAM traffic, and Cachegrind shows you exactly how much traffic each version eliminates.

7.6 KCachegrind: The Visual View

Everything above is command-line. KCachegrind is the GUI that visualizes the same data, and for exploring a complex program it is far faster than reading text. Open a Cachegrind output file directly:

```
kcachegrind naive.cg
```

bash

KCachegrind shows a sortable list of functions on the left, each with its event counts, and on the right a set of views: an annotated source view identical to `cg_annotate` but clickable, a call-map treemap where box area is proportional to cost, and (for Callgrind data) a call-graph view. You pick which event to sort by from a dropdown, so you can sort by LL read misses to immediately surface the cache-thrashing functions. For our two-line `matmul` this is overkill. For Kraken-2, with its deep call tree, being able to sort all functions by LL miss count and click straight to the worst offender's source is exactly how you find the hash-lookup hot spot in seconds.

7.7 Cachegrind on Kraken-2: The WSL2 Workaround in Practice

Now the chapter's practical payoff. On the WSL2 lab machine, `perf` cannot give you Kraken-2's LLC miss rate. Cachegrind can. Here is the exact procedure for the report.

```
# use a SMALL input – Cachegrind is ~50x slower, so a full dataset is hours
head -n 4000 reads.fastq > reads_small.fastq    # ~1000 reads

valgrind --tool=cachegrind \
  --cachegrind-out-file=kraken2.cg \
  kraken2 --db escape_db --threads 1 reads_small.fastq > out.tsv

# annotate and find the hash-lookup function's miss counts
cg_annotate kraken2.cg | grep -A3 -iE "lookup|hash|query|compact"
```

bash

Two important choices in that command. First, the small input: a thousand reads is enough to exercise the hash-lookup loop millions of times and produce a representative miss rate, while keeping the run to minutes instead of hours. The miss rate is a ratio, so it does not need a huge input to stabilize. Second, `--threads 1`: Cachegrind serializes threads anyway and single-threaded output is far cleaner to attribute. You are measuring the per-lookup cache behavior, which is identical whether one thread or thirty-two threads are doing the looking up.

What you will see is the hash-lookup function dominating the LL read miss column, with an LL data miss rate far higher than anything in our matmul benchmarks, plausibly in the range that corresponds to the 70-to-80 percent figure we expect from random probing of a multi-gigabyte table. That simulated number is what goes on the Kraken-2 report page, with a footnote that it is a Cachegrind simulation because the hardware LLC counters are unavailable under WSL2.

How This Connects to Your Project

This is the single most directly project-relevant technique in Batch 2. The Kraken-2 report page needs an LLC miss rate for the hash lookup. On WSL2 `perf` cannot provide it. Cachegrind provides it, exactly, attributed to the lookup function, reproducibly. You run it once on a small input, `grep` for the lookup function in `cg_annotate`, and copy the LL miss numbers into the report.

It is also how you prove the Hot-K-mer LRU cache works. Run Cachegrind on Kraken-2 before adding the cache, save `kraken2_before.cg`. Add the cache. Run again, save `kraken2_after.cg`. The LL read miss count on the lookup path should drop substantially, because the hot k-mers now resolve from a small structure that fits in cache instead of from a random address in the 180 GB table. The two exact integers, before and after, are the unarguable evidence that the optimization reduced DRAM traffic. `perf` could not even produce these integers on this machine; Cachegrind hands them to you deterministically.

Common Pitfall

Running Cachegrind on a full Kraken-2 dataset and waiting hours. Use a small input; the miss rate is a ratio and stabilizes quickly.

Forgetting the `-g` flag at compile time, which leaves `cg_annotate` unable to map counts to source lines, so you get function-level totals but no per-line detail.

Passing the wrong PID-suffixed filename to `cg_annotate`. Use `--cachegrind-out-file` to name the output explicitly and the problem disappears.

Reporting Cachegrind's absolute miss rate as if it were the hardware truth. It is a simulation with no prefetcher and a two-level model. State that it is simulated, and trust relative comparisons more than absolute values.

What We Just Learned

Cachegrind simulates a two-level cache (I1, D1, LL) and reports exact hit and miss counts at each level, deterministically, with no sampling.

Read the summary in three blocks: instructions (always near-perfect), data (D1 miss rate shows first-level locality), and LL (LL miss rate shows DRAM traffic). The read column dominates for numerical code.

Naive matmul shows a 33.5 percent D1 read miss rate, exactly the stride-n B access. `cg_annotate` pins it to a single source line. The `ikj` version drops the miss count by 8x, the reuse of each cache line across eight doubles.

LL misses scale toward N-cubed for naive and toward N-squared for tiled, the concrete recovery of the algorithm's true arithmetic intensity by eliminating redundant DRAM traffic.

KCachegrind visualizes the same data and lets you sort functions by any event, which is how you find the hot function in a large program like Kraken-2 quickly.

On WSL2, Cachegrind is the workaround for the LLC miss rate `perf` cannot read. Run it on a small input, single-threaded, `grep` the lookup function, and report the simulated number with a footnote.

Before You Move On

You can now produce the exact, line-attributed cache miss counts that `perf` cannot give you on WSL2, which directly fills the hardest cell of the Kraken-2 report page.

Next chapter is Callgrind, which keeps everything Cachegrind does and adds the full call graph: who called whom, how many times, and how the cache events distribute across the call tree. It also introduces `callgrind_control`, which lets you profile only the hot region of a long program like Kraken-2 and skip the expensive startup, a technique you will use to avoid simulating the 180 GB database load.

CHAPTER 8

Callgrind: Call Graph Profiling

Cachegrind told us how many cache misses each line caused. It did not tell us how the program reached that line. For our two-line matmul that is fine, because there is only one function and one caller. For Kraken-2, which calls a parser, which calls a window iterator, which calls a hash function, which calls a probe routine, the question "who called the hot function, and how often, and what did each path cost?" becomes the central question. Cachegrind cannot answer it. Callgrind can. Callgrind is Cachegrind plus the entire call graph.

Let me motivate why the call graph matters with a scenario you will actually face. Suppose perf record tells you that a small utility function, say a string-comparison or a memory-copy, is consuming twenty percent of runtime. That function is called from forty different places in the codebase. Which caller is responsible for the twenty percent? Optimizing the utility itself may be impossible (it is already optimal), but if you knew that one specific caller drove ninety percent of the calls, you could eliminate that caller's need to call it at all. The call graph is what turns "this leaf function is hot" into "this specific call path is hot, and here is where to intervene." That attribution is Callgrind's reason to exist.

8.1 What Callgrind Adds Over Cachegrind

Callgrind is built on the same instrumentation engine and the same cache simulator as Cachegrind. Everything you learned in Chapter 7 about cache counts still applies. On top of that, Callgrind records two new kinds of information.

- **The call graph itself:** for every function, which functions called it and how many times, and which functions it called. This is the directed graph of all call relationships exercised during the run.
- **Inclusive versus self cost:** for every function, the cost spent in the function itself (self cost) versus the cost spent in the function plus everything it called (inclusive cost). This distinction is the heart of call-graph analysis and we will dwell on it.

Callgrind can also, optionally, simulate the cache (the same model as Cachegrind) and the branch predictor. By default it counts instructions only, which is much faster (only about 10x to 20x overhead instead of 50x), because counting instruction executions per call edge is cheaper than simulating a cache on every access. You turn on cache simulation explicitly when you want it.

```
gcc -O2 -g -o matmul_naive matmul_naive.c

# instruction counting only – faster
valgrind --tool=callgrind \
  --callgrind-out-file=naive.callgrind \
  ./matmul_naive

# with cache simulation too – slower, but full picture
valgrind --tool=callgrind --cache-sim=yes --branch-sim=yes \
  --callgrind-out-file=naive_full.callgrind \
  ./matmul_naive
```

bash

8.2 Self Cost Versus Inclusive Cost

This single distinction confuses more people than any other concept in call-graph profiling, so I am going to be very deliberate about it. Get this right and Callgrind output reads like prose. Get it wrong and you will optimize the wrong function.

Self cost (sometimes called exclusive cost) is the cost incurred directly inside a function's own code, not counting any functions it calls. If *main* contains a loop that does some arithmetic and also calls *matmul*, then *main*'s self cost is just the arithmetic in *main*'s own body. The work done inside *matmul* is not part of *main*'s self cost.

Inclusive cost is the cost of a function plus everything it transitively calls. *main*'s inclusive cost is *main*'s own arithmetic plus all of *matmul*'s cost plus everything *matmul* calls. For the top-level *main*, the inclusive cost is essentially the whole program, because everything is reachable from *main*.

Here is the analogy that makes it stick. Think of a company's budget. A manager's *self* spending is their own salary and expenses. The manager's *inclusive* spending is their salary plus the entire budget of every team that reports to them. The CEO's self spending is small (one salary), but the CEO's inclusive spending is the entire company. If you want to cut costs, the self number tells you who personally spends a lot, and the inclusive number tells you which org branch is expensive overall. You need both to decide where to act.

Key Insight

Sort by inclusive cost to find which high-level subsystem is expensive. Sort by self cost to find which specific function to actually optimize. A function with huge inclusive cost but tiny self cost is just a dispatcher; the cost is in its children, so optimize the children. A function with large self cost is doing the work itself; that is where your hands go. Confusing the two leads people to spend days optimizing a dispatcher that does no real work.

8.3 Reading callgrind_annotate

The command-line annotator for Callgrind is `callgrind_annotate`. Its top section is a flat profile sorted by self cost, much like `perf report` but with exact integer counts.

```
callgrind_annotate naive.callgrind
```

bash

```
-----
Ir  file:function
-----
9,238,442,016  matmul_naive.c:matmul_naive [/home/ck/matmul_naive]
210,884,221   matmul_naive.c:main [/home/ck/matmul_naive]
1,043,778    ???:__memset_avx2_unaligned [/usr/lib/libc.so.6]
512,004      ???:calloc [/usr/lib/libc.so.6]
-----
```

output

Ir is the instruction-read count, Callgrind's default cost measure, a proxy for work done. The `matmul` function has 9.2 billion instruction executions as its self cost, dwarfing everything else. No surprise for this trivial program, but the format is exactly what you will read on Kraken-2, where the list will be long and the hash-lookup function will sit near the top of the self-cost column.

Below the flat profile, `callgrind_annotate` prints the call-graph section, showing for each function its callers and callees with the call counts and the cost attributed to each edge. The notation takes a moment to learn: a line marked with the cost and a `>` indicates a callee, the number in parentheses is how many times that edge was traversed.

```
-----
Ir
-----
210,884,221  main
9,238,442,016 > matmul_naive (1x)      # main called matmul_naive once,
                                     # inclusive cost 9.2 billion Ir
-----
9,238,442,016 matmul_naive
                                     # no significant callees; self == inclusive (leaf-like)
-----
```

output

This tells the whole structural story: `main` calls `matmul_naive` exactly once, and that one call carries 9.2 billion instructions of inclusive cost. `matmul_naive` has essentially no callees, so its self cost equals its inclusive cost; it is a leaf doing all its own work. For Kraken-2 this section will be rich: you will see the classification driver calling the read parser many times and the hash query many millions of times, with the cost overwhelmingly concentrated on the hash-query edge.

8.4 KCachegrind for Call Graphs

Callgrind data truly comes alive in KCachegrind, far more than Cachegrind data does, because the call graph is fundamentally a visual structure. Open it the same way:

```
kcachegrind naive.callgrind
```

bash

KCachegrind gives you views that the text annotator cannot match. The **call graph view** draws functions as boxes connected by arrows, with arrow thickness proportional to cost, so the expensive call paths are visually obvious. The **callee map** and **caller map** are treemaps where nested box area shows how inclusive cost decomposes into children, letting you see at a glance how a subsystem's cost splits among its parts. You can double-click any function to recenter the entire view on it, walking up and down the call tree interactively. For a deep program this interactive exploration finds the expensive path in seconds, where reading text annotations would take many minutes.

The practical workflow on Kraken-2: open the Callgrind output in KCachegrind, sort the function list by self cost to find the hash lookup, then switch to the caller map to see which call path drives the most calls into it, then switch to the annotated source to see the exact line. Three clicks from "something is slow" to "this line, called from this path, is the cost."

8.5 callgrind_control: Profiling Only the Hot Region

Now the feature that makes Callgrind usable on real software, and that you genuinely need for both halves of the project. The problem: Kraken-2 spends a long time at startup loading its database before it classifies a single read. Dorado spends time initializing CUDA and loading model weights. If you profile the whole run under Callgrind's 20x overhead, most of your simulation time is wasted on startup you do not care about, and the startup cost pollutes the profile. You want to profile *only* the steady-state hot loop.

callgrind_control lets you toggle profiling on and off while the program runs. You start the program with instrumentation initially disabled, let it finish its startup, then turn instrumentation on for the region you care about, then turn it off and dump the data.

```
# start with profiling OFF so startup is not instrumented
valgrind --tool=callgrind \
    --instr-atstart=no \
    --callgrind-out-file=hotloop.callgrind \
    ./matmul_naive &

# (in practice: wait until the program reports it finished initialisation)

callgrind_control -i on      # enable instrumentation NOW
# ... the hot region runs and is profiled ...
callgrind_control -i off    # disable instrumentation
callgrind_control -d        # dump the collected data to the output file
```

bash

The `--instr-atstart=no` flag is the key: it launches with the instrumentation engine present but the profiling counters dormant, so startup runs at near-Valgrind-baseline speed without recording anything. `callgrind_control -i on` arms the counters. `-i off` disarms them. `-d` dumps. You can also call these controls programmatically from inside your own code using the Callgrind macros (`CALLGRIND_START_INSTRUMENTATION` and `CALLGRIND_STOP_INSTRUMENTATION` from `valgrind/callgrind.h`), which is cleaner when you control the source and can mark the exact region.

How This Connects to Your Project

This is the technique that makes Callgrind practical on Kraken-2. The 180 GB database load is enormous and irrelevant to the classification hot loop you actually want to profile. With `--instr-atstart=no` you skip instrumenting the load entirely, then arm the counters once classification begins, capturing a clean profile of only the hash-lookup steady state. Without this you would simulate the entire database mmap and first-touch, which is both pointless and could take an extremely long time under Callgrind's overhead.

The same idea transfers conceptually to how you should profile Dorado, even though Dorado's real work is on the GPU and needs Nsight: profile the steady-state inference loop, not the one-time CUDA-context and model-weight initialization. Skip the warmup, measure the repeating work. That principle (measure steady state, exclude one-time setup) is universal across every profiler in this book.

8.6 Instruction Counting as a Deterministic Proxy

One more reason Callgrind earns its place: its default cost measure, instruction-read count (Ir), is perfectly deterministic. Run the same program on the same input under Callgrind a hundred times and you get the identical Ir count every time. `perf`'s cycle counts vary run to run because the real machine has noise: thermal effects, other processes, frequency scaling. For detecting tiny regressions this matters enormously. If a code change adds two percent more instructions, Callgrind shows exactly that two percent every time, while `perf`'s noise might be larger than the two percent you are trying to detect.

This makes Callgrind the right tool for regression testing in continuous integration, where you want a stable, reproducible number that flags when a commit increased the instruction count. The caveat is that instruction count is not the same as time: a change that adds instructions but improves cache behavior could increase Ir while decreasing wall-clock time. So Ir is a proxy for work, excellent for catching algorithmic regressions, but it is not a substitute for actual cycle measurement when memory behavior is the variable. Use Ir for "did we add work?" and `perf` cycles for "did we get slower?" They usually agree, and when they disagree, the disagreement is itself informative: it means a cache or memory effect is in play.

Common Pitfall

Optimizing a function with high inclusive cost but low self cost. That function is a dispatcher; the cost is in its children. Sort by self cost to find where the work actually happens.

Profiling startup along with the hot loop on a program like Kraken-2, polluting the profile with one-time initialization. Use `--instr-atstart=no` and `callgrind_control` to capture only steady state.

Treating Ir (instruction count) as runtime. A change can add instructions yet run faster due to better cache behavior. Ir measures work, not time; cross-check against `perf` cycles when memory behavior changes.

Forgetting that Callgrind without `--cache-sim=yes` counts instructions only and gives no cache data. If you need cache events in the call-graph context, turn cache simulation on explicitly.

What We Just Learned

Callgrind is Cachegrind plus the full call graph: who called whom, how many times, and the cost on each call edge.

Self cost is the work inside a function's own body. Inclusive cost is that plus everything it calls. Sort by inclusive to find expensive subsystems; sort by self to find the function to actually optimize.

callgrind_annotate prints a flat self-cost profile followed by a call-graph section showing caller and callee edges with their call counts and costs.

KCachegrind makes the call graph visual: a call-graph view with cost-weighted arrows, treemaps that decompose inclusive cost into children, and interactive recentering to walk the tree.

callgrind_control with --instr-atstart=no lets you skip instrumenting startup and profile only the steady-state hot loop. This is what makes Callgrind practical on Kraken-2's database-loading startup.

Callgrind's instruction count is perfectly deterministic, making it ideal for regression detection, where perf's run-to-run noise could mask the small change you are hunting for. It measures work, not time.

Before You Move On

You now have the two analysis tools of the Valgrind suite: Cachegrind for exact cache counts and Callgrind for the call-graph attribution of those counts. Together they give you a complete, deterministic, line-and-path-attributed picture of CPU behavior that no sampling profiler can match.

Next chapter steps away from performance and toward correctness and memory: Memcheck, which catches the memory bugs (use-after-free, buffer overflow, uninitialized reads, leaks) that cause crashes and silent corruption; Massif, which profiles heap growth over time and is exactly how you will understand Kraken-2's 180 GB allocation; and Helgrind, the data-race detector for the OpenMP matmul. Correctness first, because a fast program that produces wrong answers is worthless.

CHAPTER 9

Memcheck, Massif, and Helgrind

We pause the performance hunt for a chapter, because there is a kind of bug that no performance tool will ever catch and that will destroy you faster than any slow loop: a correctness bug in memory handling. A program that runs at peak speed and produces a wrong answer is worse than useless; it is dangerous, because it looks like it works. Memcheck finds the memory bugs that cause crashes and silent corruption. Massif tells you where your memory actually goes, which matters enormously when your program allocates 180 GB. Helgrind finds the data races that make threaded code produce different answers on different runs. This chapter is about trusting your results before you optimize them.

Here is the order of operations I want you to internalize for the rest of your career. Correctness first, then performance. There is no point profiling a program that is wrong. Run Memcheck, fix every error, and only then start optimizing. I have watched people spend a week tuning a kernel that had an uninitialized-memory bug feeding it garbage; the speedup was real and the answer was still wrong. Memcheck would have found it in one run.

9.1 Memcheck: What It Tracks

Memcheck is the default Valgrind tool, the one that runs when you give no `--tool/` argument. Its power comes from maintaining two pieces of shadow state for your program's entire memory, updated on every single access. Understanding these two shadow bits explains every error message Memcheck ever produces.

- **Addressability (A bits):** for every byte, is the program allowed to access it right now? A byte on the heap becomes addressable when you malloc it and unaddressable when you free it. A byte past the end of an array was never addressable. Reading or writing an unaddressable byte is an error.
- **Definedness (V bits):** for every bit, does it hold a value the program actually wrote, or is it uninitialized garbage? malloc gives you addressable but undefined memory. The moment you write to it, it becomes defined. Using an undefined value in a way that affects the program's behavior (a branch, a system call, an output) is an error.

With these two shadow states, Memcheck catches the entire family of C memory bugs. Let me name them, because each has a distinct error message you will learn to recognize.

The four bug classes

- **Invalid read/write (addressability):** touching memory you do not own. Buffer overflow past an array end, use of a dangling pointer, writing through a null pointer. Message: "Invalid read of size N" or "Invalid write of size N" with the offending line.

- **Use of uninitialized value (definedness):** branching on or outputting a value that was never written. Message: "Conditional jump or move depends on uninitialised value(s)" or "Use of uninitialised value of size N."
- **Invalid free / mismatched free:** freeing memory twice, freeing a stack pointer, freeing with the wrong deallocator (free on a new[] allocation). Message: "Invalid free()" or "Mismatched free() / delete / delete[]."
- **Memory leak:** heap memory that was allocated, is no longer reachable by any pointer, and was never freed. Found at exit when you pass --leak-check=full.

9.2 Running Memcheck

```
gcc -O0 -g -o prog prog.c      # -O0 -g gives the clearest Memcheck output bash  
  
valgrind --tool=memcheck \  
    --leak-check=full \  
    --show-leak-kinds=all \  
    --track-origins=yes \  
    ./prog
```

Three flags worth knowing. `--leak-check=full` reports each individual leak with the call stack that allocated it, not just a summary count. `--show-leak-kinds=all` includes the subtle leak categories (still-reachable, indirectly lost) that the default hides. `--track-origins=yes` is the one that saves you hours on uninitialized-value bugs: it tracks where the undefined value originally came from, so instead of "you used uninitialized memory here" you get "you used uninitialized memory here, and it was allocated uninitialized over there." That origin is usually the actual bug. It roughly doubles overhead, so enable it only when chasing a definedness error, not on every run.

A note on optimization level. For Memcheck specifically, compiling at `-O0` often gives clearer results than `-O2`, because optimizers reorder and eliminate code, which can move or mask the line attribution of an error and occasionally generate uninitialized-read false positives from legitimate optimization tricks. This is the opposite of our performance rule (always `-O2`). For correctness checking with Memcheck, `-O0 -g` is the friendlier choice. For everything performance-related, stay at `-O2`.

9.3 Memcheck on Our Benchmarks

Run Memcheck on the matmul programs and you should get a clean bill of health, because they are deliberately simple and correct. The output you want to see is:


```

==12345== HEAP SUMMARY:
==12345==      in use at exit: 0 bytes in 0 blocks
==12345==    total heap usage: 4 allocs, 4 frees, 25,165,824 bytes allocated
==12345==
==12345== All heap blocks were freed -- no leaks are possible
==12345==
==12345== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)

```

output

Four allocs (the three matrices plus libc internals), four frees, zero bytes in use at exit, zero errors. That is what correct looks like. Notice the total bytes allocated, 25,165,824, which is exactly three 1024-by-1024 double arrays at 8 MB each. Memcheck's heap summary doubles as a sanity check that your program allocated what you expected. If that number were surprising, you would have found a bug before running a single performance tool.

Now imagine a realistic bug. Suppose in `matmul` we wrote `C[i*n + j]` but forgot to `calloc` `C`, using `malloc` instead so `C` starts uninitialized, and then the accumulation `C[...] += ...` reads the uninitialized `C` before writing it. Memcheck with `--track-origins=yes` would immediately flag "Use of uninitialised value" on the accumulation line and trace the origin to the `malloc` that should have been a `calloc`. This is exactly the class of bug that produces plausible-looking but wrong matrix results, and exactly the kind a performance profiler is blind to.

9.4 Massif: Heap Profiling Over Time

Memcheck tells you about memory errors. Massif tells you about memory *consumption*: how much heap your program uses, how that usage grows and shrinks over time, and which call sites are responsible for the allocations. The output is essentially a graph of heap size against execution progress, with a breakdown of the biggest allocators at the peak.

Why does this matter for us? Because Kraken-2 allocates a 180 GB database, and understanding when and how that allocation happens is a real question for the report. Does it allocate all at once at startup, or incrementally? Is it one giant block or many? Does the resident memory match the database size, or is there overhead? Massif answers all of these by drawing the allocation curve.

```

valgrind --tool=massif \
    --massif-out-file=massif.out \
    ./prog

ms_print massif.out          # render the heap-over-time graph as text

```

bash

`ms_print` produces an ASCII chart of heap size on the vertical axis against time (measured in instructions or, optionally, bytes allocated) on the horizontal axis, with labeled snapshots at the interesting moments, especially the peak. At the peak snapshot it shows a tree of which call stacks allocated the memory, so you see not just that you used 8 GB but that 7.5 GB of it came from one specific allocation site.

The flag that matters for Kraken-2: `--pages-as-heap`

Here is a subtlety that would otherwise defeat you on Kraken-2. By default Massif only tracks heap allocations through `malloc`, `calloc`, `realloc`, and `new`. But Kraken-2 does not `malloc` its 180 GB database. It memory-maps it with `mmap`, which is a different mechanism: the OS maps the database file directly into the address space, and pages are loaded on demand as they are touched. Default Massif sees none of this, because `mmap` is not `malloc`. You would run Massif on Kraken-2 and see almost no heap usage, which is baffling until you know why.

```
valgrind --tool=massif \  
  --pages-as-heap=yes \  
  --massif-out-file=kraken2_massif.out \  
  kraken2 --db escape_db --threads 1 reads_small.fastq  
  
ms_print kraken2_massif.out
```

bash

The `--pages-as-heap=yes` flag tells Massif to track all memory mappings at the page level, including `mmap`, not just `malloc`-family allocations. Now the database mapping shows up, and you can see the memory footprint grow as Kraken-2 touches more of the database, and you can attribute the mapping to the code that performs it. This is the only way to see the true memory behavior of an `mmap`-heavy program like Kraken-2 under Massif.

How This Connects to Your Project

The report can include a real statement about Kraken-2's memory behavior backed by Massif: whether the database is mapped all at once or paged in incrementally, what the peak resident footprint is, and how it relates to the 180 GB on-disk size. Use `--pages-as-heap=yes` on a small input to see the mapping mechanism without waiting for the whole database to page in.

Massif also frames the Hot-K-mer cache discussion from the memory side. The whole motivation for the cache is that 180 GB does not fit in RAM, so the database is `mmap'd` and accessed with random page faults. Massif's page-level view shows exactly this access-driven paging. The cache works by keeping the hot k-mers in a small `malloc'd` structure that does fit in RAM, converting random page faults into local hits. Massif before and after would show the resident set of the hot structure versus the sprawling page-fault pattern of the raw `mmap`. That is a memory-side argument complementing the cache-miss argument from Chapter 7.

9.5 Helgrind: Finding Data Races

The last tool in the suite addresses a category of bug unique to threaded programs and uniquely nasty: the data race. A data race occurs when two threads access the same memory location, at least one of them writes, and there is no synchronization ordering the accesses. The result is undefined: the program may produce the right answer a thousand times and the wrong answer on the thousand-and-first, depending on the exact timing of the threads. These bugs are nearly impossible to find by testing, because they are timing-dependent and often disappear when you add the print statements you would use to debug them.

Helgrind detects races by tracking, for every memory location, which locks were held when each thread accessed it, and which happens-before relationships exist between accesses. If it finds two accesses to the same location from different threads with no lock in common and no ordering between them, and at least one is a write, it reports a race with both stack traces.

```
gcc -O2 -g -fopenmp -o matmul_omp matmul_omp.c
valgrind --tool=helgrind ./matmul_omp
```

bash

Run on our `matmul_omp` and you should see no races, because the OpenMP parallelization splits the work by rows of `C`: thread 0 writes rows 0 through `k`, thread 1 writes rows `k+1` through `2k`, and so on. Different threads write disjoint regions of `C` and only read `A` and `B`, which nobody writes. No two threads ever write the same location, so there is no race. That is correct parallel code, and Helgrind confirming zero races is exactly the reassurance you want before reporting parallel speedups.

Now imagine the bug. Suppose someone parallelized the wrong loop, putting the *omp parallel for* on the inner `j` loop instead of the outer `i` loop, so that multiple threads accumulate into the same `C[i*n + j]` simultaneously. That is a textbook data race: many threads doing `C[...] += ...` on the same element with no synchronization. The reduction is lost, results are nondeterministic, and the program produces a slightly different wrong answer each run. Helgrind would flag it instantly, naming the exact `C` element and the threads colliding on it. No performance tool would ever catch this; it is a correctness bug that only manifests under specific thread timing.

A practical caveat: OpenMP's runtime uses its own synchronization that Helgrind does not always understand, which can produce false-positive race reports about the OpenMP runtime internals rather than your code. Valgrind ships suppression files for common runtimes, and you can compile with a Helgrind-aware OpenMP if needed. When you see a reported race, check whether it is in your code or in the runtime library before acting on it. The DRD tool is an alternative race detector with a different internal algorithm that sometimes handles these cases better, so if Helgrind is noisy, try `--tool=drd` as a cross-check.

Key Insight

Data races are the only bug class in this book that can make a program produce a correct answer in testing and a wrong answer in production, purely because of thread timing. They survive testing precisely because they are nondeterministic. Helgrind and DRD are the only tools that find them by construction rather than by luck. Run one of them on any nontrivial threaded code before you trust its output, including before you report any parallel speedup from `matmul_omp`.

Common Pitfall

Profiling for performance before checking correctness. A fast wrong answer is worthless. Run Memcheck, fix everything, then optimize.

Running Massif on an mmap-heavy program like Kraken-2 without `--pages-as-heap=yes` and concluding it uses no memory. mmap is not malloc; default Massif does not see it.

Compiling at `-O2` for Memcheck and getting confusing line attribution or false uninitialized-read reports. Use `-O0 -g` for correctness checking; reserve `-O2` for performance work.

Trusting a threaded program that has never seen a race detector. The race that does not show up in a thousand test runs is the one that corrupts production output. Run Helgrind or DRD.

What We Just Learned

Correctness comes before performance. A fast program that is wrong is worse than a slow program that is right, because it hides its wrongness.

Memcheck tracks addressability (may I touch this byte?) and definedness (does this byte hold a value I wrote?) for all memory, catching invalid reads and writes, uninitialized-value use, invalid frees, and leaks. Use `--leak-check=full` and `--track-origins=yes` when chasing bugs, and `-O0 -g` for the clearest output.

Memcheck's heap summary doubles as an allocation sanity check: the total bytes should match what you expect, which can reveal a bug before any performance tool runs.

Massif profiles heap consumption over time and attributes peak usage to call sites. For mmap-heavy programs like Kraken-2 you must pass `--pages-as-heap=yes`, because the 180 GB database is mapped, not malloc'd.

Helgrind (and its alternative DRD) detect data races: unsynchronized concurrent accesses where at least one is a write. These bugs are timing-dependent and survive testing, so a race detector is the only reliable way to find them. `matmul_omp` parallelizes by disjoint rows and is race-free; parallelizing the wrong loop would race on shared C elements.

Before You Move On

You now have the full Valgrind suite: Cachegrind and Callgrind for performance analysis, Memcheck and Massif and Helgrind for correctness and memory behavior. This is a complete forensic toolkit for CPU code, exact and deterministic where perf is fast and statistical.

The next chapter is the last of Batch 2 and the oldest tool in the book: gprof, the classic Unix instrumentation profiler. It is less capable than everything we have covered, and understanding precisely why it is limited (inlining blindness, optimization interference, no thread support, no hardware counters) is itself a valuable lesson in what the modern tools fixed. It still has a place when you have no root access and no Linux perf, so it is worth knowing what it can and cannot do.

CHAPTER 10

gprof: The Classic Profiler

We close Batch 2 with the oldest profiler in this book, gprof, and I want to be honest with you from the first sentence: gprof is less capable than everything we have already learned. It cannot read hardware counters. It is blind to inlined functions. It distorts the program it measures. It does not understand threads. By every modern standard, perf and Valgrind have superseded it. So why a whole chapter? Two reasons. First, gprof is everywhere, baked into the GNU toolchain since the 1980s, and you will encounter it in legacy codebases, on locked-down machines, and in old build systems, so you must be able to read its output. Second, understanding precisely *why* gprof is limited is one of the best ways to appreciate what perf and Valgrind actually fixed. gprof is a teaching fossil: studying its limitations teaches you the design of everything that replaced it.

10.1 What gprof Is and How It Works

gprof is the GNU profiler, part of binutils. Unlike perf (which samples hardware) and Valgrind (which instruments at run time), gprof works by **compile-time instrumentation combined with sampling**, a hybrid of the two mechanisms from Chapter 1. It has two halves that together produce its output.

The first half is call counting. When you compile with the `-pg` flag, the compiler inserts a small piece of code, a call to a function named `mcount`, at the entry of every function. Each time any function is entered, `mcount` records the call and the caller. This gives gprof exact call counts and the call-graph edges: function A called function B exactly N times. This half is instrumentation, like Callgrind, but much lighter because it only hooks function entry, not every instruction.

The second half is time sampling. The `-pg` instrumentation also installs a timer that interrupts the program at a fixed rate, typically 100 times per second, and at each interrupt records which function the program counter was in. After enough samples, the fraction of samples in each function approximates the fraction of time spent there. This half is sampling, like perf record, but cruder: a fixed 100 Hz timer rather than configurable hardware-event sampling.

So gprof gives you two things from these two mechanisms: exact call counts (from `mcount` instrumentation) and approximate time distribution (from the 100 Hz timer samples). The combination lets it build a call graph annotated with both how often each edge was traversed and how much time was spent in each function. That was genuinely powerful in 1982. It is modest today.

10.2 The Three-Step Workflow

Using gprof is always the same three steps: compile with `-pg`, run the program to generate a data file, then run gprof to read it.

```
# 1. compile AND link with -pg (both matter – see the pitfalls)
gcc -O2 -pg -g -o matmul_naive matmul_naive.c

# 2. run normally – this writes a file named gmon.out in the cwd
./matmul_naive

# 3. analyze gmon.out against the binary
gprof matmul_naive gmon.out > analysis.txt
```

bash

The run produces a file named *gmon.out* (always that exact name, in the current directory) which holds the call counts and the sampling histogram. gprof reads *gmon.out* together with the binary's symbol table to produce a human-readable report. There is no live interactive mode like perf report; gprof prints a text report and you read it.

You can also turn the call graph into a picture using gprof2dot and Graphviz, which is the most legible way to consume gprof output:

```
pip install gprof2dot
gprof matmul_naive gmon.out | gprof2dot | dot -Tsvg > call_graph.svg
# open call_graph.svg in a browser
```

bash

10.3 Reading the Flat Profile

gprof output has two sections. The first is the flat profile: a table of functions sorted by the time spent in each, just like perf report but with the gprof column conventions. Here is what naive matmul produces.

Flat profile:

output

Each sample counts as 0.01 seconds.

% time	cumulative seconds	self seconds	calls	self ms/call	total ms/call	name
99.21	7.78	7.78	1	7780.00	7780.00	matmul_naive
0.51	7.82	0.04	1	40.00	40.00	main
0.28	7.84	0.02	-	-	-	frame_dummy

Read the columns carefully, because gprof's naming is specific. *% time* is the percentage of total runtime spent in this function's own code. *self seconds* is the absolute time in this function's own code. *cumulative seconds* is a running total down the list, useful for seeing how few functions account for most of the time. *calls* is the exact call count from the mcount instrumentation. *self ms/call* and *total ms/call* are per-call averages, self excluding children and total including them.

For naive matmul the story is unambiguous: matmul_naive is 99.21 percent of self time, called exactly once. The same conclusion perf and Callgrind reached, in the same shape, with the same single dominant function. The *calls* column being exactly 1 is the instrumentation half being precise, while the 99.21 percent is the sampling half being approximate. That mix of exact-counts

and approximate-time is the gprof signature.

10.4 Reading the Call Graph

The second section is the call graph, where gprof shows each function with its callers above it and its callees below it, separated into indexed entries. The format is notoriously hard to read the first time, so let me walk through the structure.

index	% time	self	children	called	name	output
		0.04	7.78	1/1	(below main)	
[1]	100.0	0.04	7.78	1	main [1]	
		7.78	0.00	1/1	matmul_naive [2]	

		7.78	0.00	1/1	main [1]	
[2]	99.2	7.78	0.00	1	matmul_naive [2]	

Each entry is bracketed by dashed lines and has an index number in brackets. The line with the index (and the function name in bold position) is the function the entry is about. Lines above it are its callers; lines below it are its callees. The *self* and *children* columns are exactly the self-versus-inclusive distinction from Chapter 8: *self* is time in the function's own code, *children* is time in everything it called. The *called* column shows the call count, often as a fraction like 1/1 meaning this caller accounted for 1 of the 1 total calls.

So entry [1] is `main`: it has self time 0.04, children time 7.78 (all in `matmul_naive`), and it called `matmul_naive` once. Entry [2] is `matmul_naive`: called once by `main`, self time 7.78, no children. This is the same call-graph information Callgrind gave us, in a clunkier text format. If you learned to read Callgrind's caller/callee sections in Chapter 8, this is the same concept with different formatting.

10.5 Why gprof Is Limited: The Important Part

Now the real lesson of the chapter. gprof has four serious limitations, and each one teaches you something about why the modern tools were built the way they are.

Limitation 1: inlined functions are invisible

This is the big one. When you compile at `-O2` or `-O3`, the compiler inlines small functions, pasting their bodies directly into their callers to eliminate call overhead. But gprof's mcount instrumentation hooks function *calls*, and an inlined function is never called, its body is just spliced in. So inlined functions vanish from gprof's profile entirely; their time is silently attributed to whatever function they were inlined into. On modern optimized code, where aggressive inlining is the norm, this means gprof's profile can be wildly misleading, blaming a caller for time actually spent in inlined callees.

The workaround is ugly: to make gprof see your functions, you have to defeat the optimization that makes them fast, compiling with `-O0` or `-fno-inline`. But then you are profiling a different, slower

program with different behavior. This is the fundamental gprof bind: it either lies about your fast code (by hiding inlined functions) or measures slow code (by disabling inlining). perf and Valgrind escape this entirely because they work on the actual machine code as it runs, inlining and all, mapping samples back to source via debug info rather than hooking calls.

Limitation 2: -pg distorts the measured program

The -pg instrumentation adds an mcount call at every function entry. That call has a cost, and it changes the program you are measuring. Functions called very frequently pay a relatively large instrumentation overhead, which can distort their apparent cost. You are not measuring your program; you are measuring your program plus mcount calls. perf's near-zero overhead and Valgrind's uniform overhead both avoid this specific distortion-of-relative-costs problem.

Limitation 3: no thread support

gprof was designed for single-threaded programs. Its sampling timer and its gmon.out accounting do not correctly handle multiple threads; on a threaded program it typically profiles only the main thread or produces garbage for the others. Our matmul_omp, with its eight OpenMP threads, is exactly the case gprof cannot handle. perf, Callgrind, and the others all support threads properly. If your code is threaded, gprof is simply the wrong tool.

Limitation 4: no hardware counters

gprof measures time and call counts. It cannot tell you cache misses, branch mispredictions, IPC, or anything about *why* a function is slow. It answers only the *where* question from Chapter 1, never the *why*. For our entire memory-bound matmul analysis, gprof would tell you matmul_naive is the hotspot and stop, leaving you with no idea that the cause is cache misses. perf stat and Cachegrind exist precisely to answer the why that gprof cannot.

Key Insight

Every gprof limitation maps to a feature of a modern tool. Inlining blindness, fixed by perf and Valgrind working on real machine code. Distortion from instrumentation, fixed by perf's zero overhead. No threads, fixed by everything modern. No hardware counters, fixed by perf stat and Cachegrind. gprof is a museum of the problems that the rest of this book's tools were built to solve. Knowing gprof's weaknesses is knowing why the modern toolkit looks the way it does.

10.6 When gprof Is Still the Right Choice

Despite all that, gprof is not useless, and I want to be fair to it. There are real situations where it is the right or only tool.

- **No root access.** perf often needs elevated privileges to read hardware counters (the perf_event_paranoid setting from Chapter 3). On a shared cluster or a locked-down machine where you cannot change that and have no sudo, gprof needs nothing special: you compile with -pg and run as a normal user.

- **Non-Linux platforms.** perf is Linux-specific. On other Unix systems or in cross-compilation toolchains for embedded targets, gprof (or its vendor equivalents) may be the only profiler available.
- **Quick single-threaded function-level hotspots.** If you just want to know which function dominates a simple single-threaded program and you do not care why, gprof's three-command workflow is fast and requires no setup.
- **Legacy build systems.** Some old projects have -pg profiling wired into their makefiles already. Reading the existing gprof output is faster than introducing a new tool.

How This Connects to Your Project

For the Nanopore pipeline specifically, gprof is mostly the wrong tool, and knowing why is itself useful. Kraken-2 is heavily threaded, so gprof cannot profile it correctly; you use perf and Cachegrind instead. Dorado's real work is on the GPU, which gprof cannot see at all; you use Nsight. And the whole point of the project is the *why* (cache misses, memory bandwidth), which gprof cannot answer.

Where gprof could earn a cameo: if you write a small single-threaded preprocessing or analysis utility around the pipeline, on a machine where you lack the privileges to run perf, gprof's no-setup workflow would give you a quick function-level hotspot read. But for the core profiling that goes in the report, gprof's four limitations rule it out. The honest report uses perf, Cachegrind, and Nsight. Recognizing that gprof does not fit here is the correct application of everything this chapter taught.

Common Pitfall

Forgetting -pg at link time. The flag is needed at BOTH compile and link steps; if you compile object files with -pg but link without it, the mcount machinery is incomplete and gmon.out is never written or is empty.

Profiling an -O2/-O3 build and trusting the result despite inlining. Inlined functions vanish from gprof. Either accept the distortion or compile with -fno-inline (and accept measuring slower code).

Running gprof on a threaded program and believing the numbers. gprof does not handle threads; the output reflects at most one thread.

Expecting cache or branch data. gprof has none. It answers where, never why. For why, you need perf stat or Cachegrind.

What We Just Learned

gprof is a hybrid profiler: compile-time `-pg` instrumentation gives exact call counts via `mcount`, and a 100 Hz timer gives approximate time distribution. The workflow is always compile with `-pg`, run to make `gmon.out`, analyze with `gprof`.

The flat profile lists functions by self time with exact call counts. The call-graph section shows callers and callees with the same self-versus-inclusive distinction Callgrind uses.

gprof has four serious limitations: it is blind to inlined functions (catastrophic on optimized code), it distorts the program via instrumentation overhead, it does not support threads, and it has no hardware counters so it cannot answer why.

Each limitation maps to something a modern tool fixed. gprof is a teaching fossil that shows you the problems `perf` and `Valgrind` were designed to solve.

gprof still earns its place when you lack root for `perf`, when you are off Linux, or when you need a quick single-threaded function-level read with no setup. For the threaded, GPU-bound, why-focused Nanopore pipeline, it is the wrong tool, and knowing that is the right call.

Before You Move On

You have now completed Batch 2 and the entire CPU profiling toolkit. `perf` for fast statistical hardware truth, the `Valgrind` suite for exact deterministic forensics, and `gprof` as the classic fallback whose limitations illuminate the rest. For any CPU program you can find the hotspot, explain the bottleneck, verify correctness, understand memory consumption, detect races, and quantify improvements.

Batch 3 crosses the PCIe bus to the GPU. Everything changes: a different memory hierarchy, thousands of threads in warps, streaming multiprocessors, and an entirely different set of bottlenecks (occupancy, coalescing, bank conflicts, warp stalls). The philosophy carries over (measure, find the bottleneck, understand why, quantify the ceiling), but the tools are NVIDIA's `Nsight Systems` and `Nsight Compute`, and the workload is `Dorado`. We begin with a GPU architecture review, the Chapter 2 of the GPU world, because just as you could not read `perf` without knowing the CPU, you cannot read `Nsight` without knowing the GPU.